

EE/CS 172/130
Digital Logic and Design

Topic 2 – Binary Codes

Dr. Maria Waqas

Binary Codes

- An n -bit binary code is a group of n bits that can assume up to 2^n distinct combinations of 1's and 0's.
- Each unique combination represents one element of the set being coded.
- A set of 4 elements can be represented using 2 bits: Possible codes $\rightarrow$ 00, 01, 10, 11
- A set of 8 elements requires 3 bits: Possible codes $\rightarrow$ from 000 to 111
- A set of 16 elements requires 4 bits: Possible codes $\rightarrow$ from 0000 to 1111
- In general, the bit combinations of an n -bit code are obtained by counting in binary from 0 to $2^n - 1$.
- **Rules**
 - Each element must be assigned a unique binary combination.
 - No two elements can share the same code, otherwise the code assignment becomes ambiguous.

Straight Binary Codes

- When a decimal number is represented directly by its equivalent binary number, it is called a **straight binary code**.
- **Disadvantage:** Conversion between decimal and binary can become long and complicated for large numbers.
- This makes straight binary representation unsuitable for certain applications.
- For such cases, special coding schemes are used that combine features of both decimal and binary systems.

Weighted Codes

- In a **weighted code system**, each digit position is assigned a specific weight, and the decimal value of the code is obtained by taking the algebraic sum of the weights corresponding to the positions where a **1** appears.
- **Examples of weighted codes**
 - Straight binary code
 - 8421 code - most commonly used, also known as BCD – Binary Coded Decimal
 - 2421 code
 - 7421 code
 - $74\bar{2}\bar{1}$ code
 - $84\bar{2}\bar{1}$ code
- **Examples of unweighted codes**
 - Excess-3
 - Gray Code
 - ASCII Code
 - Parity Code

Binary Coded Decimal (BCD)

- Also called 8421 BCD.
- In this system, each decimal digit (0–9) is represented by its equivalent 4-bit binary code.
- The designation 8-4-2-1 indicates the binary weights of the four bit positions: $2^3 = 8$, $2^2 = 4$, $2^1 = 2$, $2^0 = 1$
- Since only digits 0–9 are valid in decimal, 8421 BCD uses only the first 10 combinations of the 16 possible 4-bit patterns. The unused codes (1010–1111) are considered *invalid* in BCD.
- Invalid BCD codes:
 - 1010
 - 1011
 - 1100
 - 1101
 - 1110
 - 1111

Decimal Symbol	BCD Digit
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Binary Coded Decimal (BCD)

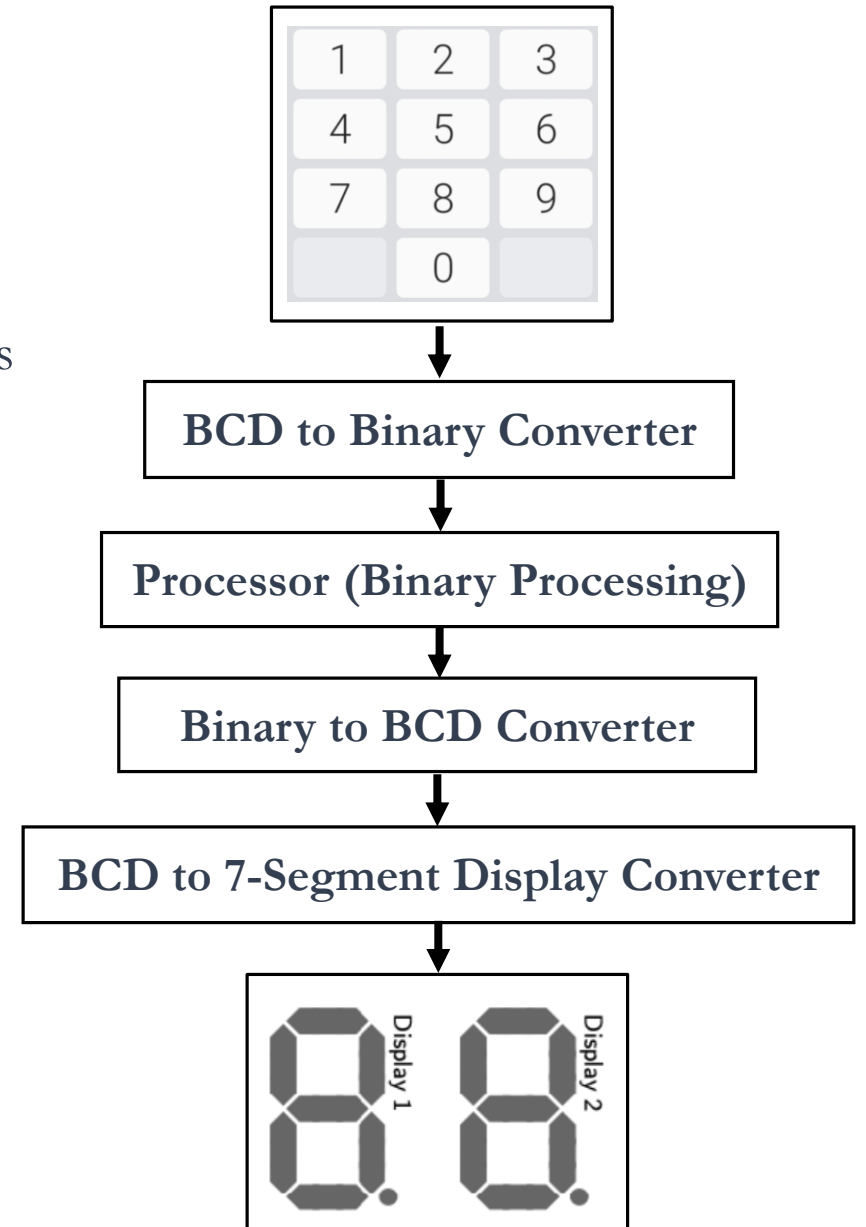
Applications of 8421 BCD

Keypads (Input Devices)

- In calculators, telephones, and digital systems, keypads use decimal digits (0–9).
- When a key is pressed, the decimal digit is encoded into its 8421 BCD form.
- This BCD code is then sent to the processor or controller for further operations.

Displays (Output Devices)

- Seven-segment LED or LCD displays cannot directly understand binary numbers.
- Instead, they use a BCD-to-Seven-Segment Decoder/Driver.
- This is why 8421 BCD is often called a **bridge code** between binary systems and human-friendly decimal digits.



BCD Arithmetic

Rules for Addition of BCD Values

1. Add the BCD number using the rules for binary addition.
 2. Start scanning the 4-bit groups from right.
 - **Case A:** If the 4-bit sum is equal to or less than 9:
 - Leave the result as it is; it is a valid BCD number.
 - **Case B:** If the 4-bit sum is greater than 9 but less than 15, i.e. no carry is generated:
 - Add 0110_2 to the 4-bit sum in order to skip the six invalid states.
 - If a carry results when 0110_2 is added, simply add the carry to the next 4-bit group.
 - **Case C:** If the 4-bit sum is greater than 9 and generates a carry:
 - Add this carry to the next 4-bit group.
 - Add 0110_2 to the 4-bit sum.
-
- **Examples:** Add the following in BCD and verify the given results.
 - $450 + 417 = 867$
 - $16 + 15 = 31$
 - $67 + 53 = 120$
 - $38 + 48 = 86$

BCD Arithmetic

Rules for Subtraction of BCD Values (Using 10's Complement):

- Take the 10's complement of the **subtrahend** (the value to be subtracted).
- Convert both the **minuend** (value from which subtraction is done) and the 10's complement of the subtrahend into BCD, then add them.
- If a carry out of the most significant digit (MSD) is generated, discard the carry. The result is positive and already in BCD.
- If no carry is generated, the result is negative. In this case, take the 10's complement of the obtained sum (in BCD) and prefix a negative sign.
- **Examples:** Perform the following subtractions in BCD and verify the given results.
 - $78 - 29 = 53$
 - $325 - 016 = 309$
- BCD is not a self-complementing code, so we cannot directly take the 10's complement of a BCD pattern.

Other Decimal Codes

Decimal Digit	BCD 8421	2421	Excess-3	8, 4, -2, -1
0	0000	0000	0011	0000
1	0001	0001	0100	0111
2	0010	0010	0101	0110
3	0011	0011	0110	0101
4	0100	0100	0111	0100
5	0101	1011	1000	1011
6	0110	1100	1001	1010
7	0111	1101	1010	1001
8	1000	1110	1011	1000
9	1001	1111	1100	1111
Unused bit combi- nations	1010	0101	0000	0001
	1011	0110	0001	0010
	1100	0111	0010	0011
	1101	1000	1101	1100
	1110	1001	1110	1101
	1111	1010	1111	1110

- The unused codes (invalid states) can be used for [error detection](#).

Gray Code

- A binary coding system where **only one bit changes** when moving from one number to the next in sequence.
 - Consecutive numbers differ in only **one bit position**.
- Also called **Unit Distance Code** or **Unit Change Code**.
- No fixed weights assigned to bit positions (unweighted).
- Useful in systems where errors may occur if multiple bits change simultaneously.

Applications

- **Input/Output Devices:** Reduces errors in digital communication.
- **Instrumentation:** Common in rotary encoders to measure shaft position.
- **Data Acquisition Systems:** Prevents misreading when converting analog signals to digital.

Gray Code	Decimal Equivalent
0000	0
0001	1
0011	2
0010	3
0110	4
0111	5
0101	6
0100	7
1100	8
1101	9
1111	10
1110	11
1010	12
1011	13
1001	14
1000	15

Gray Code

- Let the binary value be represented by $B_1B_2B_3\dots B_n$ and its gray equivalent be $G_1G_2G_3\dots G_n$, where B_1 and G_1 are MSBs.

- Binary to Gray Conversion**

$$G_1 = B_1$$

$$G_2 = B_1 \oplus B_2$$

$$G_3 = B_2 \oplus B_3$$

.

.

$$G_n = B_{n-1} \oplus B_n$$

- Gray to Binary Conversion**

$$B_1 = G_1$$

$$B_2 = B_1 \oplus G_2$$

$$B_3 = B_2 \oplus G_3$$

.

.

$$B_n = B_{n-1} \oplus G_n$$

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Example

- $22)_{10} = 10110)_2 = 11101)_G$

Example

- $11101)_G = 10110)_2 = 22)_{10}$

Alphanumeric Codes

- Many applications of digital computers require handling not only numbers but also letters and special symbols.

ASCII (American Standard Code for Information Interchange)

- 7-bit code → represents 128 characters (0–127).
- Includes: control characters, digits, uppercase/lowercase letters, punctuation.

- **Examples**

- Character 'A' → Decimal 65 → Binary 1000001
- Character 'a' → Decimal 97 → Binary 1100001
- Character '0' → Decimal 48 → Binary 0110000

Char	Dec	Oct	Hex	Char	Dec	Oct	Hex	Char	Dec	Oct	Hex
(sp)	32	0040	0x20	@	64	0100	0x40	`	96	0140	0x60
!	33	0041	0x21	A	65	0101	0x41	a	97	0141	0x61
"	34	0042	0x22	B	66	0102	0x42	b	98	0142	0x62
#	35	0043	0x23	C	67	0103	0x43	c	99	0143	0x63
\$	36	0044	0x24	D	68	0104	0x44	d	100	0144	0x64
%	37	0045	0x25	E	69	0105	0x45	e	101	0145	0x65
&	38	0046	0x26	F	70	0106	0x46	f	102	0146	0x66
'	39	0047	0x27	G	71	0107	0x47	g	103	0147	0x67
(	40	0050	0x28	H	72	0110	0x48	h	104	0150	0x68
)	41	0051	0x29	I	73	0111	0x49	i	105	0151	0x69
*	42	0052	0x2a	J	74	0112	0x4a	j	106	0152	0x6a
+	43	0053	0x2b	K	75	0113	0x4b	k	107	0153	0x6b
,	44	0054	0x2c	L	76	0114	0x4c	l	108	0154	0x6c
-	45	0055	0x2d	M	77	0115	0x4d	m	109	0155	0x6d
.	46	0056	0x2e	N	78	0116	0x4e	n	110	0156	0x6e
/	47	0057	0x2f	O	79	0117	0x4f	o	111	0157	0x6f
0	48	0060	0x30	P	80	0120	0x50	p	112	0160	0x70
1	49	0061	0x31	Q	81	0121	0x51	q	113	0161	0x71
2	50	0062	0x32	R	82	0122	0x52	r	114	0162	0x72
3	51	0063	0x33	S	83	0123	0x53	s	115	0163	0x73
4	52	0064	0x34	T	84	0124	0x54	t	116	0164	0x74
5	53	0065	0x35	U	85	0125	0x55	u	117	0165	0x75
6	54	0066	0x36	V	86	0126	0x56	v	118	0166	0x76
7	55	0067	0x37	W	87	0127	0x57	w	119	0167	0x77
8	56	0070	0x38	X	88	0130	0x58	x	120	0170	0x78
9	57	0071	0x39	Y	89	0131	0x59	y	121	0171	0x79
:	58	0072	0x3a	Z	90	0132	0x5a	z	122	0172	0x7a
;	59	0073	0x3b	[	91	0133	0x5b	{	123	0173	0x7b
<	60	0074	0x3c	\	92	0134	0x5c		124	0174	0x7c
=	61	0075	0x3d	]	93	0135	0x5d	}	125	0175	0x7d
>	62	0076	0x3e	^	94	0136	0x5e	~	126	0176	0x7e
?	63	0077	0x3f	_	95	0137	0x5f				

Alphanumeric Codes

Extended ASCII

- 8-bit code → represents 256 characters (0–255).
- First 128 are the same as standard ASCII.
- The extra 128 characters (128–255) are used for graphics, accented letters, and special symbols (useful for European languages).
- **Examples**
 - Character ‘Ç’ (C with cedilla) → Decimal 128 (in extended ASCII table)
 - Character ‘é’ (e with acute) → Decimal 130
 - Character ‘⦿’ (block drawing symbol) → Decimal 176

Alphanumeric Codes

Unicode

- Also called Universal Code.
- Modern universal coding standard.
- Supports over 1.1 million characters, covering all world languages, symbols, emojis, and scripts.
- Variable length: UTF-8 (1–4 bytes), UTF-16 (2 or 4 bytes), UTF-32 (4 bytes).
- **Examples** (Unicode code points)
 - Character ‘A’ → U+0041
 - Character ‘♥’ (heart symbol) → U+2665
 - Character ‘अ’ (Hindi letter A) → U+0905
 - Character ‘你’ (Chinese character for “you”) → U+4F60
 - Emoji ‘😊’ → U+1F60A

Error Detection and Correction Codes

- When data is transmitted, it is (like all electrical systems) susceptible to noise that can alter or distort the contents.
 - A 1 may be flipped into a 0, and vice versa.
- To ensure reliable communication, special error detection and correction codes are used.

Parity Code

- Adds a single parity bit to each character.
- Detects single-bit errors.
- Cannot correct errors.
- Used in serial communication, memory systems, and low-cost embedded systems.
- Types of parity systems:
 - **Even parity:** The parity bit is chosen so that the total number of 1's in the data + parity is even.
 - **Odd parity:** The parity bit is chosen so that the total number of 1's in the data + parity is odd.
- A given system (transmitter + receiver) can operate with either even or odd parity but not both.
- Odd parity is generally preferred over even parity:
 - If all data bits become 0 due to a fault (e.g., short circuit), then with even parity the total number of 1's = 0, which is still even → error goes undetected.
 - With odd parity, this condition is easily detected (since 0 ones $\neq$ odd).

Error Detection Codes

Hamming Code

- Adds multiple redundant bits placed at positions that are powers of 2 (1, 2, 4, 8...).
- Can detect up to 2-bit errors.
- Can correct 1-bit errors automatically (without retransmission).
- Used in memory systems (ECC RAM) and communication systems.

Checksum

- Data is divided into fixed-size blocks (words).
- All blocks are added together, and the sum (checksum) is appended to the message.
- At the receiving end, the sum is recalculated.
- Detects errors by mismatch of calculated vs transmitted checksum.
- Commonly used in network protocols (TCP/IP, UDP).